SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Restaurant App MVP (Frontend Only)

Student Name: *ZAINAB FARHAN*

Student ID / Roll No: *9327*

Course*: Mobile App Development (Fall 2026)*

Instructor*: MISS SADAF TANVIR*

1. INTRODUCTION

1.1 Purpose

This document defines the requirements for the Restaurant App MVP (v1.0.0), a frontend-only React Native application designed for Diners and Managers.

1.2 Scope

- In-Scope: Authentication, daily specials, menu browsing, debounced search, shopping cart, promo discounts, table reservations, live order tracking, and Manager Dashboard.

- Out-of-Scope: Backend databases, real payments, push notifications, SMS OTP, and GPS mapping.

1.3 Definitions and Acronyms

SRS (Software Requirements Specification): A structured document outlining all operational features, constraints, and expectations of a software application.

MVP (Minimum Viable Product): A streamlined version of a software product containing only core features required to deliver value to early users.

UML (Unified Modeling Language): A standardized visual modeling language used to visualize, specify, and document system software components.

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

Hook: A specialized React function allowing functional components to hook into state management and component lifecycle events.

Context: A React mechanism allowing global state data to be accessed across deep component hierarchies without explicit prop drilling.

Reducer: A pure function accepting the current state and an action payload to produce an updated state without side effects.

Mock Data: Local static JSON or JavaScript object arrays mimicking a live server database response.

FlatList: A high-performance React Native component designed to efficiently render long, scrollable lists of structured data using windowing techniques.

2. OVERALL DESCRIPTION

2.1 User Roles

- Customer: Browses menu, applies promos, places orders, books tables, and tracks order status.

- Manager: Manages menu prices/stock, reviews reservations, and overrides order statuses.

2.2 User Stories

1. As a customer, I want to view daily specials to quickly find discounted dishes.

2. As a customer, I want to filter dishes by category to navigate faster.

3. As a customer, I want debounced search to avoid UI lag while typing.

4. As a customer, I want to add special instructions to cart items for customized food.

5. As a customer, I want to apply promo codes for percentage discounts.

6. As a customer, I want to reserve tables by date, time, and party size to avoid waiting.

7. As a customer, I want a live progress timer to know when my order is ready.

8. As a manager, I want to toggle dish stock status to prevent invalid orders.

9. As a manager, I want to accept or decline incoming table reservations.

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

10. As a manager, I want to update menu prices directly in the dashboard.

3. FUNCTIONAL REQUIREMENTS

Authentication Module:

- FR-01: System shall authenticate users against local mock datasets with 1-second delay.

- FR-02: System shall validate email format, password strength (min 8 chars, 1 digit), and password matching.

- FR-03: System shall route Customers to Menu and Managers to Dashboard upon login.

Menu & Search Module:

- FR-04: System shall render at least 15 items across Starters, Mains, Desserts, and Drinks.

- FR-05: System shall highlight daily specials with a visual badge.

- FR-06: System shall grey out out-of-stock items and disable their Add buttons.

- FR-07: System shall debounce search inputs by 400ms.

- FR-08: System shall save the last 5 search queries as tappable suggestions.

- FR-09: System shall allow sorting items by Price and Name.

Cart & Reservation Module:

- FR-10: System shall allow quantity modification, item removal, and special notes in cart.

- FR-11: System shall validate promo codes (WELCOME10, FEAST20) and apply discounts.

- FR-12: System shall calculate subtotal, 5% service charge, 15% tax, and grand total.

- FR-13: System shall validate phone numbers against Pakistani format (03XX-XXXXXXX).

- FR-14: System shall disable hourly slots (12:00-22:00) if table capacity is insufficient.

- FR-15: System shall reject past reservation dates and require 1-hour advance booking.

Orders & Dashboard Module:

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

- **FR-16:** System shall generate unique order records with initial status "Pending".

- **FR-17:** System shall automatically advance status (Pending -> Preparing -> Ready -> Served) using timers.

- **FR-18:** System shall show a live elapsed time counter during order tracking.

- **FR-19:** System shall restrict Manager Dashboard tab to Manager role only.

- **FR-20:** System shall persist manager menu/order edits to AsyncStorage.

4. NON-FUNCTIONAL REQUIREMENTS

- **Usability:** Clean UI adhering to Apple & Material guidelines with min 48dp touch targets.

- **Performance:** Screen transitions < 300ms, FlatList scrolling at 60 FPS.

- **Responsiveness:** Responsive design supporting screens from 320px width to tablets.

- **Maintainability:** Modular code structured into components, screens, hooks, and reducers.

- **Data Handling:** Client-side storage persistence using @react-native-async-storage/async-storage.

5. CLIENT-SIDE DATA MODEL

Data Set Name	Fields	Description
Users	id, name, email, password, role	Holds registered account credentials and role designations (customer vs manager).
Categories	id, name, icon	Defines the menu classification groupings.

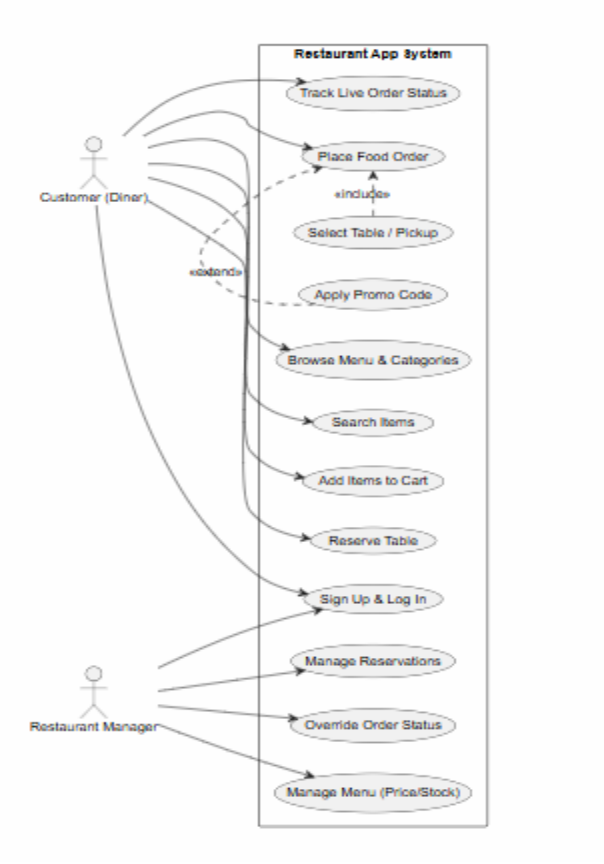
BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

Data Set Name	Fields	Description
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	Stores the complete catalog of dishes, pricing, and current stock status.
Tables	id, tableNumber, capacity, isReserved	Represents physical dining tables available for reservation.
Reservations	id, userId, date, timeSlot, partySize, tableId, phone, status	Contains active and past table bookings submitted by customers.
Orders	id, userId, items, subtotal, tax, discount, grandTotal, type, status, timestamp	Tracks customer food orders, item breakdowns, total costs, and current preparation state.

6. UML DIAGRAMS

- Use Case Diagram: Customer & Manager interactions across 12 use cases.

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

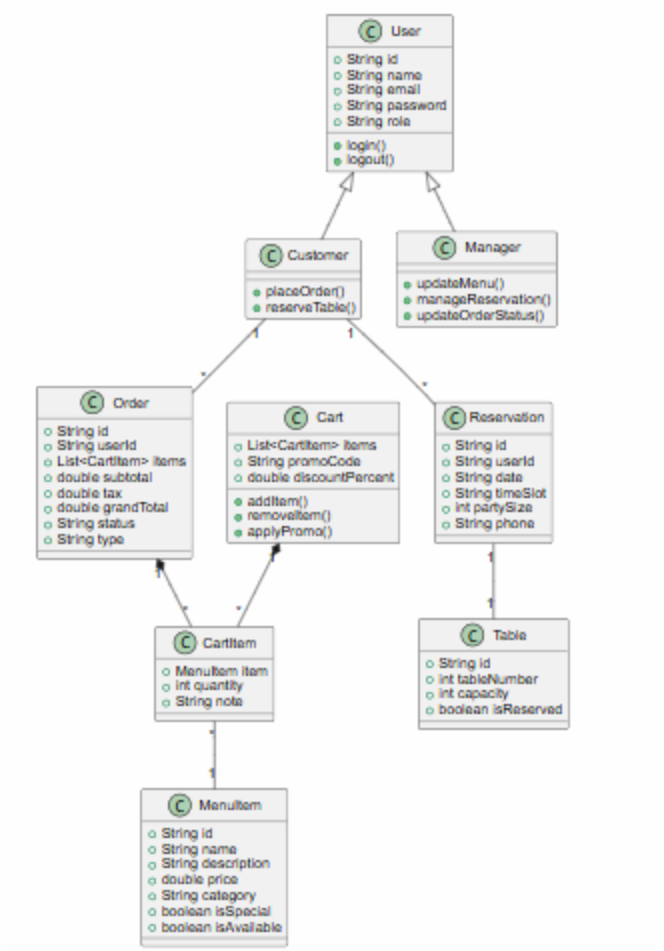


Explanation:

This diagram illustrates the functional boundaries of the application and models direct interactions between the Customer and Manager actors across 12 core features, including promo code extensions and table selection dependencies.

- Class Diagram: Data structures and multiplicities (User, Order, Reservation, Table, Cart).

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

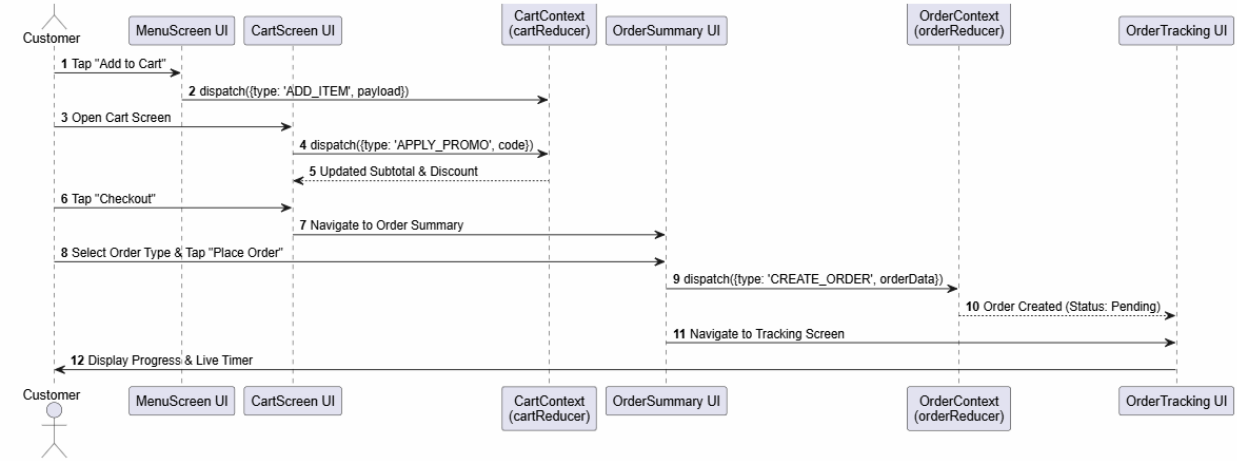


Explanation:

This diagram outlines the structural entities of the system—such as User, MenuItem, Cart, Order, Reservation, and Table—along with their internal attributes, method signatures, inheritance rules, and relational multiplicities.

- Sequence Diagram: Customer placing an order to tracking lifecycle.

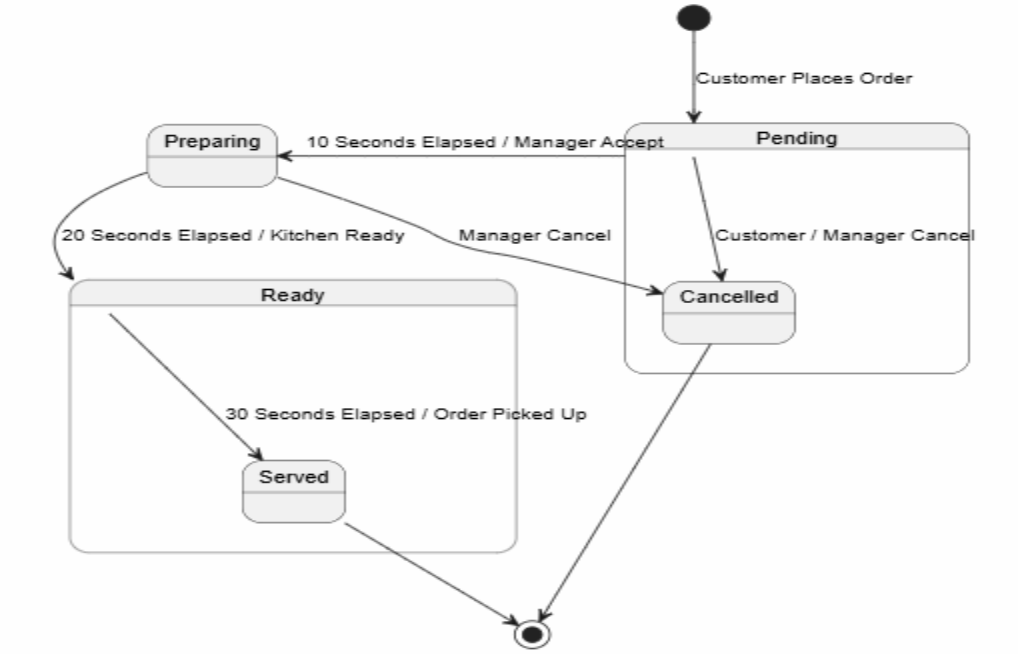
BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.



Explanation:

This diagram traces the step-by-step message execution flow for placing an order, detailing how user actions move from adding items on the menu screen to dispatching cart updates and opening the live order tracking view.

- State Machine Diagram: Order lifecycle states (Pending, Preparing, Ready, Served, Cancelled).

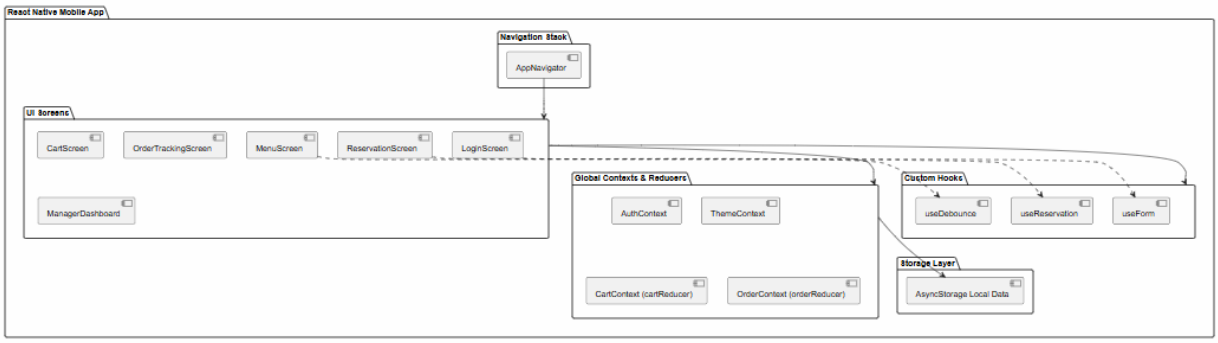


BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

Explanation:

This diagram maps the lifecycle transitions of a food order from Pending through Preparing, Ready, and Served, highlighting automatic background timer triggers as well as manual cancellation paths.

- Component Diagram: Dependencies between Navigation, Contexts, Hooks, and Screens.



Explanation:

This diagram depicts the high-level software architecture, showing the dependencies between AppNavigator, UI screens, shared Context Providers, pure state reducers, custom hooks, and persistent local storage.

7. MVP FRONTEND DEVELOPMENT STRUCTURE

7.1. Login and Signup Screen

- **Purpose:** Authenticates existing users or registers new customer/manager accounts.
- **UI Elements:** Mode toggle buttons, Email input, Password input, Confirm Password input, Role Selector picker, Submit button with ActivityIndicator, Show Password toggle, Error labels.
- **Navigation Entry/Exit:** Entry: App Launch / Logout. Exit: MainTabs (for Customers) or ManagerDashboard (for Managers).
- **Local Data Used:** mockUsers array from src/data/users.js.

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

- **Hooks Used & Why:** useState to manage active view mode, submission flags, and password visibility; custom useForm hook to handle controlled inputs, field validation, and dynamic error clearing.

7.2. Menu Browsing Screen

- **Purpose:** Displays daily specials, dish listings categorized by section, and allows adding items to the cart.
- **UI Elements:** Horizontal category chips, dish cards (MenuItemCard), Daily Special badges, Price tags, Add to Cart buttons, Pull-to-refresh spinner.
- **Navigation Entry/Exit:** Entry: Bottom Tab Bar ("Menu"). Exit: Cart Screen, Item Detail Modal.
- **Local Data Used:** initialMenuItems array from src/data/menu.js.
- **Hooks Used & Why:** useState to track the active category; useEffect to simulate initial data fetching delays and set header titles; useCallback to prevent unnecessary re-rendering of menu item cards when items are added to cart.

7.3. Search and Scroll Controls

- **Purpose:** Enables real-time searching across dishes with input debouncing, recent search history, and smooth scroll navigation.
- **UI Elements:** Search bar input with clear button, search query suggestion chips, floating "Back to Top" scroll button, render counter debug label.
- **Navigation Entry/Exit:** Integrated directly into MenuScreen.js.
- **Local Data Used:** searchHistory state array storing the last 5 unique search queries.
- **Hooks Used & Why:** Custom useDebounce hook to delay search filter calculations by 400ms to avoid UI stutter; useRef to store search bar focus references, timer IDs, FlatList scroll pointers, and render counters without triggering re-renders.

7.4. Profile and Theme Screen

- **Purpose:** Displays user profile details, role permissions, and global light/dark theme toggles.
- **UI Elements:** User avatar, Name/Email/Role labels, Light/Dark mode toggle switch, Logout button.

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

- **Navigation Entry/Exit:** Entry: Bottom Tab Bar ("Profile"). Exit: LoginScreen (upon logout).
- **Local Data Used:** Authenticated user session object from AuthContext.
- **Hooks Used & Why:** Custom useAuth hook to access current user data and logout function; custom useTheme hook to retrieve active theme colors and toggle global styling mode via React Context.

7.5. Cart Screen

- **Purpose:** Reviews selected dishes, adjusts quantities, appends special preparation instructions, and validates promo codes.
- **UI Elements:** Item quantity increment/decrement buttons, special instruction text inputs, promo code field with "Apply" button, subtotal label.
- **Navigation Entry/Exit:** Entry: Menu Screen / Tab Bar ("Cart"). Exit: OrderSummaryScreen.
- **Local Data Used:** cartState object from CartContext.
- **Hooks Used & Why:** useReducer inside CartContext to handle complex state operations (ADD_ITEM, INCREMENT, DECREMENT, UPDATE_NOTE, APPLY_PROMO) through predictable, pure state actions.

7.6. Order Summary Screen

- **Purpose:** Presents a complete financial cost breakdown and order type selection before checkout.
- **UI Elements:** Itemized cost table, Service charge (5%) label, Sales tax (15%) label, Promo discount deduction label, Order Type toggle (Dine-in vs. Takeaway), "Place Order" button.
- **Navigation Entry/Exit:** Entry: Cart Screen. Exit: OrderTrackingScreen.
- **Local Data Used:** Cart items and applied discount values from CartContext.
- **Hooks Used & Why:** useMemo to memoize financial calculations (subtotal, tax, service charges, and grand total) so they are only recalculated when cart items or discount percentages change.

7.7. Table Reservation Screen

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

- **Purpose:** Enables customers to book dining tables based on party size, date, and hourly time slots.
- **UI Elements:** Date picker calendar, time slot selector buttons (12:00–22:00), party size stepper (1–12), Pakistani phone number input field, "Reserve Table" button, confirmation modal.
- **Navigation Entry/Exit:** Entry: Bottom Tab Bar ("Reserve"). Exit: Booking Confirmation Modal / My Reservations List.
- **Local Data Used:** mockTables array from src/data/tables.js.
- **Hooks Used & Why:** Custom useReservation hook to encapsulate table availability logic, validate phone numbers against local regex format, and manage reservation submissions.

7.8. Order Tracking and Manager Dashboard

- **Purpose:** Provides live order status progress timers for customers and management controls for restaurant managers.
- **UI Elements:**
    - *Customer View:* Step progress indicator bar, live elapsed seconds counter, status status labels.
    - *Manager View:* Incoming orders list with status override buttons, table reservation approval controls, menu item editor modal.
- **Navigation Entry/Exit:** Entry: Order Placement / Manager Tab. Exit: Main Navigation Stack.
- **Local Data Used:** orders, reservations, and menuItems stored in Contexts and AsyncStorage.
- **Hooks Used & Why:** useEffect to manage setInterval timers for tracking elapsed seconds and triggering automatic status transitions; AsyncStorage hooks to persist menu and order modifications locally on the client device.

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.